



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN



Linear Attention Gated DeltaNet & Kimi Delta Attention

IDEA-ISAIL Reading Group

Roadmap



1. **Linear attention: fixed-size memory**
2. GDN and KDA: better memory updates
3. Chunkwise DeltaNet: efficient parallelism
4. Model quality and decoding efficiency



Softmax attention and quadratic work



Every query compares against its entire causal prefix.

$$Q, K, V \in \mathbb{R}^{N \times d}, \quad O = \text{softmax}_{\text{causal}} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

$$1 + 2 + \dots + N = \frac{N(N+1)}{2} \implies O(N^2d) \text{ work}$$

Can a compact memory replace repeated reads of the full history?



Linear attention by associativity



Replace softmax weights with a dot-product kernel.

$$o_t = \sum_{j \leq t} (q_t^\top k_j) v_j^\top = q_t^\top \underbrace{\left(\sum_{j \leq t} k_j v_j^\top \right)}_{S_t : \text{memory matrix}}$$

$$S_t = S_{t-1} + k_t v_t^\top, \quad o_t = q_t^\top S_t$$

Each new token updates and reads one $d \times d$ matrix.



Fixed-size memory changes the scaling



Core operator, per head. Sequence length N and dimension d .

Quantity	Softmax	Linear recurrence
Full sequence work	$O(N^2d)$	$O(Nd^2)$
One decode step	$O(Nd)$	$O(d^2)$
Decode cache / state	$O(Nd)$	$O(d^2)$

At fixed d , the state and each decode step stay constant in N .



Attention Is All You Need. Vaswani et al., NIPS 2017.

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. Katharopoulos et al., ICML 2020.

Parallelizing Linear Transformers with the Delta Rule over Sequence Length. Yang et al., NeurIPS 2024.

Roadmap



1. Linear attention: fixed-size memory
- 2. GDN and KDA: better memory updates**
3. Chunkwise DeltaNet: efficient parallelism
4. Model quality and decoding efficiency



DeltaNet: correction instead of accumulation



A fixed-size memory needs a way to revise stored associations.

$$\underbrace{k_t^\top S_{t-1}}_{\text{current prediction}} \quad \underbrace{v_t^\top - k_t^\top S_{t-1}}_{\text{prediction error}}$$

$$S_t = S_{t-1} + \beta_t k_t \left(v_t^\top - k_t^\top S_{t-1} \right)$$

β_t controls the correction. The readout remains $o_t = q_t^\top S_t$.

DeltaNet as online gradient descent



Fast memory S learns one key-value association at each token.

$$\ell_t(S) = \frac{1}{2} \left\| \underbrace{k_t^\top S}_{\text{prediction}} - \underbrace{v_t^\top}_{\text{target}} \right\|_2^2$$

$$\nabla_S \ell_t(S) = k_t(k_t^\top S - v_t^\top)$$

$$\begin{aligned} S_t &= S_{t-1} - \beta_t \nabla_S \ell_t(S_{t-1}) \\ &= \boxed{S_{t-1} + \beta_t k_t(v_t^\top - k_t^\top S_{t-1})}. \end{aligned}$$

One online step on S per token. β_t is the step size.

GDN: forgetting and correction



Scalar decay frees memory before the new association is written.

$$\bar{S}_t = \alpha_t S_{t-1}, \quad S_t = \bar{S}_t + \beta_t k_t (v_t^\top - k_t^\top \bar{S}_t)$$

$$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

α_t : global retention β_t : correction at the current key

Both gates come from the input and can be computed in advance.

KDA: forgetting by feature



Different memory features can retain information for different times.

$$D_t = \text{diag}(\alpha_t), \quad \alpha_t \in (0, 1)^d$$

$$S_t = (I - \beta_t k_t k_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$$

GDN	KDA
One decay per head	One decay per key feature
$\alpha_t I$	$D_t = \text{diag}(\alpha_t)$



Linear state transitions enable reordering



Once the inputs are known, each update is affine in the memory.

$$S_t = A_t S_{t-1} + B_t, \quad o_t = q_t^\top S_t$$

$$\text{GDN: } A_t = \alpha_t (I - \beta_t k_t k_t^\top)$$

$$\text{KDA: } A_t = (I - \beta_t k_t k_t^\top) D_t$$

$$B_t = \beta_t k_t v_t^\top$$

Compose known update rules, then apply them to the incoming state.



Parallelizing Linear Transformers with the Delta Rule over Sequence Length. Yang et al., NeurIPS 2024.

Gated Delta Networks: Improving Mamba2 with Delta Rule. Yang et al., ICLR 2025.

Kimi Linear: An Expressive, Efficient Attention Architecture. Zhang et al. (Kimi Team), arXiv technical report, 2025.

Roadmap



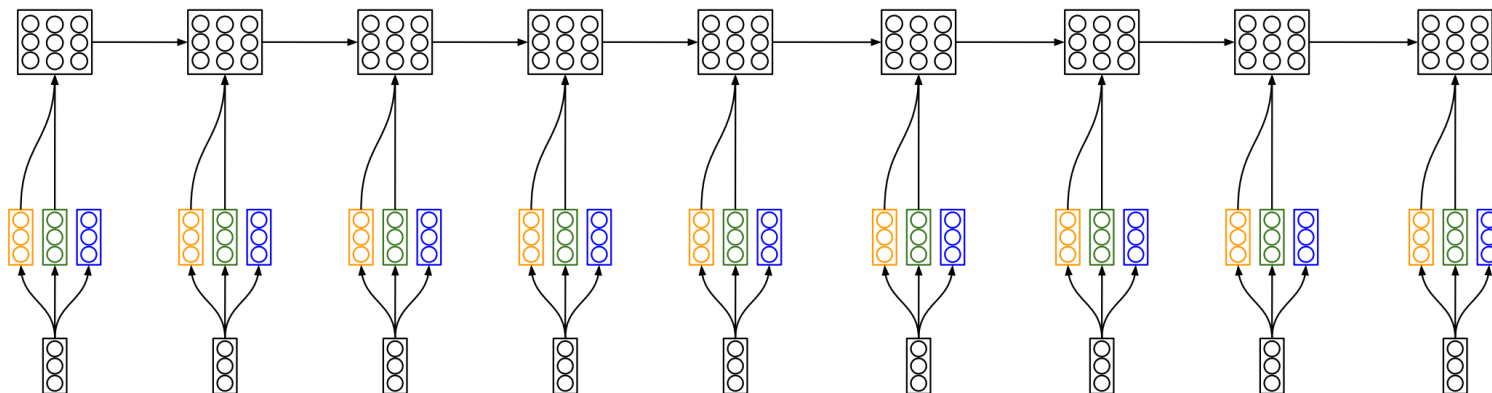
1. Linear attention: fixed-size memory
2. GDN and KDA: better memory updates
- 3. Chunkwise DeltaNet: efficient parallelism**
4. Model quality and decoding efficiency



The parallelization goal



Training / prefill: compact memory with batched matmul.



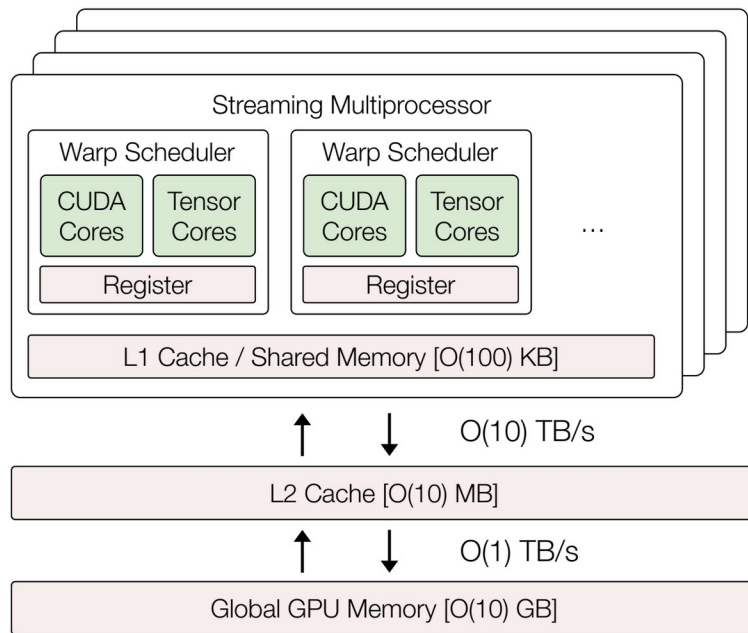
Token recurrence: N dependent steps

Chunkwise: N / C boundary steps

Prepare each chunk independently. Read all its outputs together.



Why GPUs favor matrix multiplication



Tensor Cores

Specialized units for small matrix multiply-accumulate operations.

Reuse data on chip

A matrix tile serves many operations before returning to GPU memory.

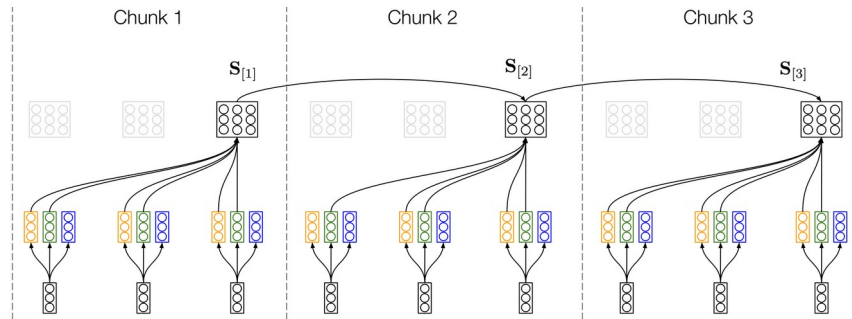
Chunking trades some extra arithmetic for reuse and parallel work.



Simple linear attention: the chunk we want



One boundary state preserves history; local writes combine by matmul.



$$S_{\text{out}} = S_{\text{in}} + \underbrace{K^T V}_{\text{all } C \text{ local writes}}$$

Within this chunk: Q, K, V are $C \times d$. The state is $d \times d$.

Chunk outputs depend on incoming memory



Sequential boundary schedule: pass one state between chunks.

$$S_{\text{in}}^{(b)} = S_{\text{out}}^{(b-1)}, \quad S_{\text{out}}^{(b)} = S_{\text{in}}^{(b)} + K_b^{\top} V_b$$

The output matrix contains all C token outputs of chunk b.

$$O_b = \underbrace{Q_b S_{\text{in}}^{(b)}}_{\text{needs incoming memory}} + \underbrace{\text{tril}(Q_b K_b^{\top}) V_b}_{\text{local causal contribution}}$$

The local term can be prepared before the boundary state arrives.

Why not parallelize the entire sequence?



Choose enough parallel work without creating a global score matrix.

Additive operator	Work	Tradeoff
Token recurrence	$O(Nd^2)$	Long dependency chain
Full masked QK^T	$O(N^2d)$	All token pairs
Chunkwise	$O(Nd^2 + NCd)$	Local $C \times C$ interactions

Storing every token state S_t costs $O(Nd^2)$ space.

Chunking stores boundary states and reconstructs local outputs.



Additive chunk boundaries by prefix scan



Each chunk computes its increment without incoming memory.

$$\Delta_b = K_b^\top V_b, \quad S_{\text{in}}^{(b)} = S_0 + \underbrace{\sum_{j < b} \Delta_j}_{\text{exclusive prefix sum}}$$

Exclusive means all preceding chunks, excluding the current chunk.

$$(\Delta_1, \Delta_2, \Delta_3, \dots) \mapsto (0, \Delta_1, \Delta_1 + \Delta_2, \dots)$$

Associative addition permits a parallel tree computation.

All chunks can finish their outputs in parallel once the states are ready.

DeltaNet writes depend on the old state



Corrected write values for a chunk of C tokens

$$e_t^\top = \beta_t \left(v_t^\top - k_t^\top S_{t-1} \right)$$

Target output

$$O = Q S_{\text{in}} + \text{tril}(Q K^\top) E$$

$$E = \begin{bmatrix} e_1^\top \\ \vdots \\ e_C^\top \end{bmatrix} \in \mathbb{R}^{C \times d}$$

$$\Delta S_t = k_t e_t^\top = \beta_t k_t \left(v_t^\top - k_t^\top S_{t-1} \right)$$

We cannot precompute fixed increments for an additive scan.

Next: separate incoming-state dependence from local computation.

A chunk does two things to memory



Transform incoming history, and add locally generated content.

$$M_i = I - \beta_i k_i k_i^\top, \quad X_i = \beta_i k_i v_i^\top$$

$$S_1 = M_1 S_{\text{in}} + X_1,$$

$$S_2 = \underbrace{M_2 M_1}_{P_2} S_{\text{in}} + \underbrace{M_2 X_1 + X_2}_{H_2}.$$

$$\boxed{S_i = P_i S_{\text{in}} + H_i}$$

$$P_i = M_i P_{i-1},$$

$$H_i = M_i H_{i-1} + X_i.$$

$P_0 = I$ and $H_0 = 0$. Both summaries use only this chunk's inputs.

The WY representation of the transition product



Original: compose the state transitions in their temporal order.

$$\begin{aligned} P_i &= M_i M_{i-1} \cdots M_1, & P_0 &= I \\ &= (I - \beta_i k_i k_i^\top) \cdots (I - \beta_1 k_1 k_1^\top). \end{aligned}$$

Lemma: WY representation for DeltaNet transitions

$$\boxed{P_i = I - \sum_{j=1}^i k_j w_j^\top}, \quad w_j = \beta_j P_{j-1}^\top k_j$$

Each factor is identity minus rank one, a generalized Householder form.



Deriving the WY vectors



Induction: assume the representation holds for the previous prefix.

$$\begin{aligned} P_i &= (I - \beta_i k_i k_i^\top) P_{i-1} \\ &= P_{i-1} - k_i \underbrace{\beta_i k_i^\top P_{i-1}}_{w_i^\top} \\ &= I - \sum_{j < i} k_j w_j^\top - k_i w_i^\top. \end{aligned}$$

$$w_i = \beta_i \left(k_i - \sum_{j < i} (k_i^\top k_j) w_j \right)$$

$$P_C = I - K^\top W, \quad W = [w_1, \dots, w_C]^\top \in \mathbb{R}^{C \times d}$$



Compress the local contribution into U



The zero-initial-state contribution has the same key directions.

$$H_i = H_{i-1} + k_i \underbrace{\beta_i (v_i^\top - k_i^\top H_{i-1})}_{u_i^\top}$$

$$H_i = \sum_{j \leq i} k_j u_j^\top, \quad H_C = K^\top U$$

$$u_i = \beta_i \left(v_i - \sum_{j < i} (k_i^\top k_j) u_j \right)$$

U and W share the same feedback coefficients; only their inputs differ.

Deriving the actual write values



Substitute the two summaries into the state decomposition.

$$P_i = I - \sum_{j \leq i} k_j w_j^\top, \quad H_i = \sum_{j \leq i} k_j u_j^\top$$

$$S_i = P_i S_{\text{in}} + H_i$$

$$= S_{\text{in}} - \sum_{j \leq i} k_j w_j^\top S_{\text{in}} + \sum_{j \leq i} k_j u_j^\top$$

$$= S_{\text{in}} + \sum_{j \leq i} k_j \underbrace{\left(u_j^\top - w_j^\top S_{\text{in}} \right)}_{e_j^\top}.$$

$$i = C : \quad S_{\text{out}} = S_C = S_{\text{in}} + K^\top E$$

Actual write values and the chunk state



Correct the zero-start write values for incoming memory.

$$E = U - W S_{\text{in}}$$

$$e_j^T = u_j^T - w_j^T S_{\text{in}}, \quad j = 1, \dots, C$$

K, U, W and E each store one row per token ($C \times d$).

$$S_{\text{out}} = S_{\text{in}} + K^T E$$

Compute the actual write values E, then update the boundary memory.



One shared system prepares U and W



Move each earlier-vector correction to the left.

$$w_i^\top + \sum_{j < i} \beta_i (k_i^\top k_j) w_j^\top = \beta_i k_i^\top,$$

$$u_i^\top + \sum_{j < i} \beta_i (k_i^\top k_j) u_j^\top = \beta_i v_i^\top.$$

$$B = \text{diag}(\beta), \quad R = I + \text{tril}(BK K^\top, -1)$$

$$\boxed{RW = BK, \quad RU = BV}$$

The local transform absorbs the recurrence



Compute one transform, then apply it to both sets of inputs.

$$T = R^{-1}, \quad \boxed{W = TBK, \quad U = TBV}$$

$$R = \begin{bmatrix} 1 & 0 & 0 \\ g_{21} & 1 & 0 \\ g_{31} & g_{32} & 1 \end{bmatrix}, \quad g_{ij} = \beta_i(k_i^\top k_j) \quad (j < i)$$

All chunks can build their own T, U and W independently.



What T means: all causal feedback paths



Repeated substitution combines direct and indirect corrections.

$$A = -\text{tril}(BKK^{\top}, -1), \quad A^C = 0, \quad R = I - A$$

$$T = (I - A)^{-1} = I + A + A^2 + \dots + A^{C-1}$$

$$T_{31} = \underbrace{a_{31}}_{1 \rightarrow 3} + \underbrace{a_{32}a_{21}}_{1 \rightarrow 2 \rightarrow 3}$$

Paths have at most $C - 1$ edges, so the series terminates exactly.



How to compute T with matrix operations



Solve small diagonal blocks, then combine their interactions.

$$R = \begin{bmatrix} R_{11} & 0 \\ R_{21} & R_{22} \end{bmatrix}, \quad T = \begin{bmatrix} T_{11} & 0 \\ -T_{22}R_{21}T_{11} & T_{22} \end{bmatrix}$$

$$T_{11} = R_{11}^{-1}, \quad T_{22} = R_{22}^{-1}$$

Diagonal blocks: independent local triangular work.

Cross-block correction: matmul. Many chunks run at once.



Back to the goal: the same chunk structure



The entire derivation produces a replacement for additive writes V.

$$\underbrace{W = TBK, \quad U = TBV}_{\text{prepare without } S_{\text{in}}} \quad \underbrace{E = U - WS_{\text{in}}}_{\text{when history arrives}}$$

$$S_{\text{out}} = S_{\text{in}} + K^{\top} E$$

$$O = QS_{\text{in}} + \text{tril}(QK^{\top})E$$

What runs in parallel, and what still waits?



Phase	What it computes	Execution
1. Prepare chunks	R, T, U and W No incoming state needed	All chunks in parallel
2. Pass memory	$E = U - WS_{in}$ $S_{out} = S_{in} + K^T E$	Chunks in order Heads / batch parallel
3. Read outputs	$O = QS_{in} + \text{tril}(QK^T)E$	All chunks in parallel

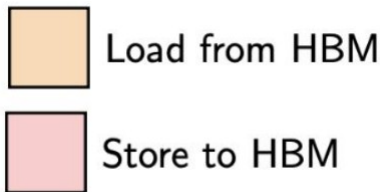
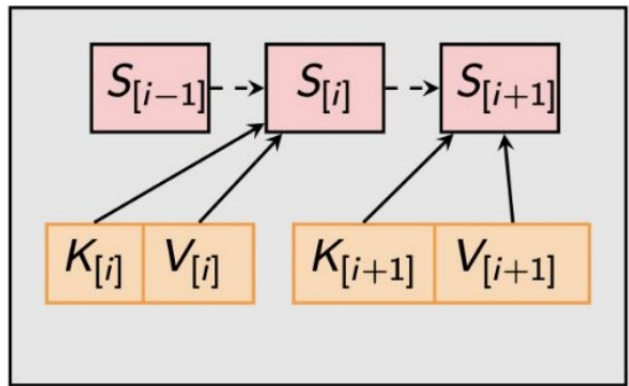
A shorter boundary chain connects large blocks of parallel work.



Kernel design: reuse a tile across many tokens



Boundary memory

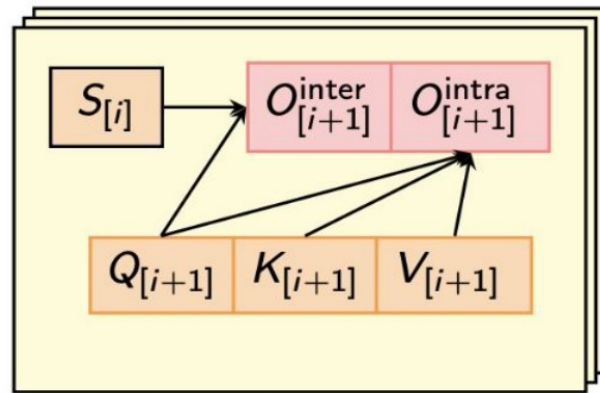


Keep state tiles on chip

Apply C rows, then save a boundary.

Data reuse and shorter dependencies both contribute to speed.

Chunk outputs



Reconstruct token outputs

Read history and local score tiles.

The cost of the chunkwise schedule



Work across all chunks	Cost	Source
State reads / updates	$O(Nd^2)$	$C \times d$ times $d \times d$
Local interactions	$O(NCd)$	$C \times C$ score transforms
Local triangular preparation	$O(NC^2)$	Dense blocked cost bound

$$C \lesssim d \implies O(Nd^2 + NCd) \text{ total work}$$

N / C boundary steps; $O((N / C)d^2)$ space for saved states.

Roadmap



1. Linear attention: fixed-size memory
2. GDN and KDA: better memory updates
3. Chunkwise DeltaNet: efficient parallelism
- 4. Model quality and decoding efficiency**



GDN improves language modeling and reasoning



1.3B parameters, 100B tokens, 4K context. Matched training recipe.

Metric	Transformer++	GDN	GDN + SWA
WikiText perplexity ↓	18.53	16.42	16.07
Zero-shot average ↑	52.25	55.32	56.40
Retrieval average ↑	37.0	30.6	39.0

Hybrid H1 adds sliding-window attention and improves retrieval.



Kimi Linear often improves quality



48B total / 3B active. Same 1.4T tokens and training recipe.

Kimi Linear uses 3 KDA layers per 1 full-attention MLA layer.

Evaluation	Full MLA	Kimi Linear
Base: MMLU-Pro ↑	47.2	51.0
Base: MMLU ↑	71.6	73.8
128K: RULER ↑	81.3	84.3
128K: LongBench v2 ↑	36.1	35.0
128K: 8-task average ↑	52.2	54.5

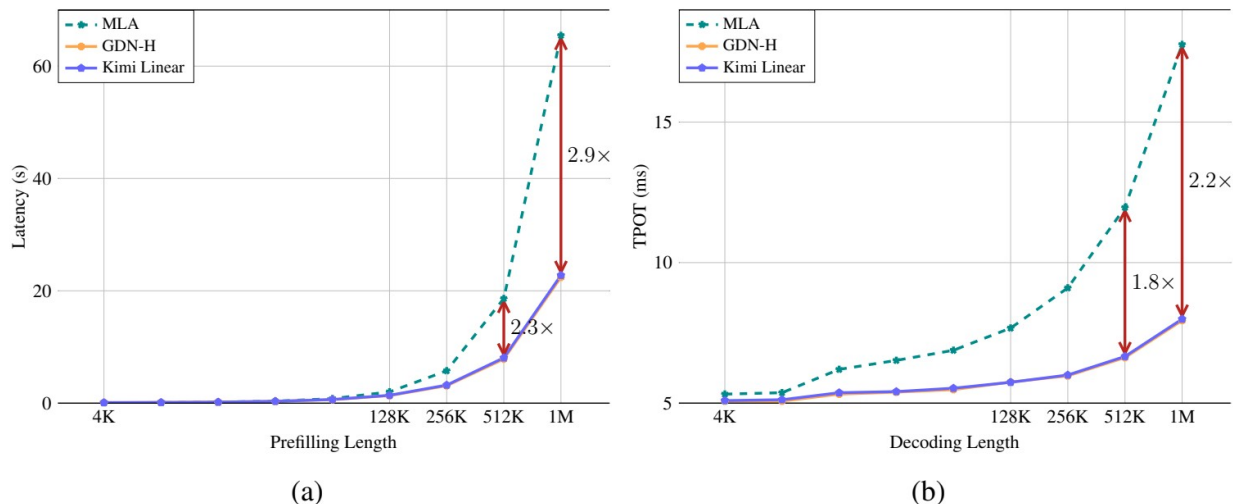
Broad gains in matched runs, with task-dependent tradeoffs.



Faster inference at long context



Matched model configurations, batch 1. Baseline: full MLA.



At 1M context: 2.9× faster prefill and about 2.2× faster decode.



The design principles



Compact state makes long-history decoding affordable.

Correction and forgetting make that state more useful.

Chunking turns known inputs into reusable matrix operations.

Where do we still need full attention?

Exact retrieval, memory capacity and the hybrid ratio.



Appendix: the full GDN layer



Input H generates Q , K , V , α , β and an output gate.

Stage	Computation
Prepare	Input norm, projections, short causal convolution Q/K feature normalization and input-dependent gates
Memory operator	$(Q, K, V, \alpha, \beta, S_0)$ gives token outputs O and final state S_N
Finish	Output norm and gate, concatenate heads, output projection

Y is passed to the residual block. S_N is retained as recurrent memory.

Appendix: GDN: where the decay factors enter



$$\gamma_i = \prod_{r=1}^i \alpha_r, \quad g_{ij} = \prod_{r=j+1}^i \alpha_r \quad (j \leq i)$$

$$S_i = \gamma_i S_{\text{in}} + \sum_{j \leq i} g_{ij} k_j e_j^{\top}$$

$$(Q_{\text{in}})_i = \gamma_i q_i^{\top}, \quad (K_{\text{end}})_j = g_{Cj} k_j^{\top}$$

$$G_{ij} = \begin{cases} g_{ij}, & j \leq i, \\ 0, & j > i, \end{cases} \quad A_{\text{local}} = G \odot (QK^{\top})$$

Each write decays only after it enters memory.

Appendix: GDN: solve for the effective writes



The correction reads the decayed memory before writing.

$$e_i^\top = \beta_i \left(v_i^\top - \alpha_i k_i^\top S_{i-1} \right)$$

$$\alpha_i k_i^\top S_{i-1} = \gamma_i k_i^\top S_{\text{in}} + \sum_{j < i} g_{ij} (k_i^\top k_j) e_j^\top$$

$$e_i^\top + \sum_{j < i} \beta_i g_{ij} (k_i^\top k_j) e_j^\top = \beta_i \left(v_i^\top - \gamma_i k_i^\top S_{\text{in}} \right)$$

All feedback coefficients are known from the chunk inputs.

Appendix: The triangular system gives U and W



$$B = \text{diag}(\beta), \quad \Gamma = \text{diag}(\gamma), \quad R = I + \text{tril}\left(B[(KK^\top) \odot G], -1\right)$$

$\text{tril}(\cdot, -1)$ keeps only earlier-token interactions.

$$RE = B(V - \Gamma K S_{\text{in}}) \quad R \in \mathbb{R}^{C \times C}$$

$$\underbrace{RU = BV}_{\text{local writes}},$$

$$\underbrace{RW = B\Gamma K}_{\text{response to incoming memory}}$$

$$E = U - WS_{\text{in}}$$



Appendix: GDN: the target chunkwise form



If we can prepare effective writes E , the rest is matmul.

$$\boxed{E = U - W S_{\text{in}}} \in \mathbb{R}^{C \times d}$$

$$S_{\text{out}} = \gamma_C S_{\text{in}} + K_{\text{end}}^T E$$

$$O = Q_{\text{in}} S_{\text{in}} + A_{\text{local}} E$$

U and W use only local inputs. E stacks the actual corrected writes.



Appendix: KDA retains the same chunkwise template



Replace scalar decay paths with feature-wise decay paths.

$$P_{i:j} = D_i D_{i-1} \cdots D_{j+1}, \quad P_{i:i} = I$$

$$\underbrace{g_{ij}(k_i^\top k_j)}_{\text{GDN interaction}} \longrightarrow \underbrace{k_i^\top P_{i:j} k_j}_{\text{KDA interaction}}$$

Prepare local coefficients. Pass boundary memory. Compute outputs.

Feature scaling preserves a structured triangular problem.

Appendix: KDA: feature scaling exposes matmul



Cumulative feature gates factor each pairwise interaction.

$$p_i = \prod_{r \leq i} \alpha_r, \quad K_+ = \Pi \odot K, \quad K_- = K \oslash \Pi, \quad Q_+ = \Pi \odot Q$$

$$R = I + B \operatorname{tril}(K_+ K_-^\top, -1), \quad A_{\text{local}} = \operatorname{tril}(Q_+ K_-^\top)$$

The same local solve and boundary schedule can be reused.

Π stacks p_i by row. Products and division above are elementwise.



Appendix: complete KDA chunk form



$$h_{ij} = k_i^\top P_{i:j} k_j, \quad F_i = k_i^\top P_{i:0}, \quad R = I + B \operatorname{tril}(H, -1)$$

$$RU = BV, \quad RW = BF, \quad E = U - W S_{\text{in}}$$

$$S_{\text{out}} = P_{C:0} S_{\text{in}} + \sum_{j=1}^C P_{C:j} k_j e_j^\top$$

$$o_i = q_i^\top P_{i:0} S_{\text{in}} + \sum_{j \leq i} (q_i^\top P_{i:j} k_j) e_j^\top$$



Appendix: KDA interactions as matmul



$$P_i = D_i \cdots D_1, \quad \hat{k}_i = P_i k_i, \quad \check{k}_j = P_j^{-1} k_j, \quad \hat{q}_i = P_i q_i$$

$$k_i^\top P_{i:j} k_j = \hat{k}_i^\top \check{k}_j, \quad q_i^\top P_{i:j} k_j = \hat{q}_i^\top \check{k}_j$$

$$H = \text{tril}(\hat{K} \check{K}^\top), \quad A_{\text{local}} = \text{tril}(\hat{Q} \check{K}^\top)$$

Kernel implementations use stable gate differences and subchunks.



Appendix: gate parameterization



$$\beta_t = \sigma(w_\beta^\top x_t + b_\beta)$$

$$\delta_t = \text{softplus}(w_\alpha^\top x_t + b_\alpha), \quad \alpha_t = \exp[-\exp(a)\delta_t]$$

Positive rate and step size give retention between zero and one.

KDA learns a retention value for each key feature.



References



Parallelizing Linear Transformers with the Delta Rule over Sequence Length

Yang et al., NeurIPS 2024.

Gated Delta Networks: Improving Mamba2 with Delta Rule

Yang et al., ICLR 2025.

Kimi Linear: An Expressive, Efficient Attention Architecture

Zhang et al. (Kimi Team), arXiv technical report, 2025.

The WY Representation for Products of Householder Matrices

Bischof and Van Loan, SIAM J. Sci. Stat. Comput., 1987.

Additional references and source URLs appear in the slide notes.

